Lesson notes · for you

Fine-tuning and guardrails

AI agents · University · Lesson 4

Time	Materials
20 min	Paper and a pen; Your phone, charged or plugged in, for the lab (a training run can take up to about 11 minutes on an old phone)

Learning goals

- Measure what fine-tuning teaches and what it makes a model forget.
- Explain why learned safety is not a guarantee.
- Design layered guardrails in code: tools, approval, checks, logs, tests.

Activity: Activity: Layers of defence

1. Pick (3 min): In pairs, choose an agent (a study-group assistant, a club helper) and one risky action it could take, such as messaging everyone or booking a room.
2. Layers (7 min): Write five guards on separate cards: the fewest tools, approval of the exact action, a detail check, a log, a re-test after changes. Under each, write what it stops.
3. Attack (7 min): Your partner describes a failure (the model rewords a refusal, invents a message after “yes”, forgets after a fine-tune) and removes one card at a time. Decide after each removal whether the action would still be stopped.
4. Debrief (3 min): Agree which guard mattered most, and which failure no single card would stop on its own.

Use made-up agents, people and messages only. Five index cards or torn paper squares are enough.

Discussion

- When is fine-tuning worth it, and when is a new tool or a clearer instruction enough?
- Why is a model’s promise to ask first not enough?
- Which guardrail would you never remove, and why?

Check yourself

- One thing fine-tuning can make a model forget, and the guard in code that still protects you.

Try it this week

Write a one-page guardrail plan for an agent you would like to build: its fewest tools, the actions that need approval of the exact action, the checks in code, what it logs, and the tests you will re-run after every change.

Worksheet

Fine-tuning and guardrails

AI agents · University · Lesson 4

Name _____ Class _____ Date _____

1 **A model has learned to ask before it sends a message. Why still check the approval in code?**

- ◯ Code runs faster than the model
- ◯ Learned behaviour can fail on new wordings or after fine-tuning
- ◯ The model can't send messages anyway

2 **Write gate(call, approved, log). It appends (tool, ok) to log, where ok says whether call is exactly the approved call, and returns "run" only then; otherwise it returns "ask again". approved is None when nothing has been approved. (No model here: the calls are given.)**

```
def gate(call, approved, log):
    return "run"
```

Write your program:

3 **Match each teach-lab run to what Darsi Tiny scores on 8 bus requests and 8 old skills.**

Before teaching	• • • •	Bus 5/8, old skills 8/8
No old conversations mixed in	• • • •	Bus 7/8, old skills 5/8
16 old conversations mixed in	• • • •	Bus 7/8, old skills 1/8
48 old conversations mixed in	• • • •	Bus 0/8, old skills 8/8

4 **This approval gate has one line that lets a different message through after a single yes. Which line?**

Circle the line with the mistake. Write it correctly:

```
1  def gate(call, shown, reply, log):
2      if reply != "yes":
3          return "ask"
4      log.append(call["tool"])
5      if call["tool"] == shown["tool"]:
6          return "run"
7      return "ask"
```

5 **Which of these keep protecting you even if the model's learned safety fails? Choose all that apply.**

- [ ] An allow-list of tools in code for each task
- [ ] Comparing each call with the approved call in code
- [ ] A line in the prompt: "Always ask before sending."
- [ ] A log of every action, allowed or refused
- [ ] Trusting the refusals it learned in training

✎ **Think of an agent you would like to build or use. Which of its behaviours must hold every time, and which layer of code would guarantee each one?**